

EXPERIMENT-16(HDOC)

C Programming

Menu-Driven Program: Largest/Smallest Rearrangement, Digit Swap and Palindrome Check

Step 1: Understand the Problem Statement

- Input: A positive integer, and a menu choice selecting the operation to perform.
- Output: Depending on the choice — the largest possible number formed by rearranging its digits, the smallest possible number formed by rearranging its digits, the number obtained after swapping its first and last digits, or whether the number reads the same forwards and backwards (a palindrome).
- This is a menu-driven program using switch-case, where each option calls its own separate void function (with no parameters, no return value, and no call by value/reference — a simple call): generateLargest(), generateSmallest(), swapFirstLast(), and checkPalindrome(). Each function independently reads the number, processes it, and prints the result.
- For the largest/smallest operations, the digits are extracted into an array and sorted (descending for largest, ascending for smallest) before being reassembled into a number. For swapping, the digits are extracted, reversed to restore left-to-right order, and the first and last positions are exchanged.

Rule used for palindrome: A number is a palindrome if reversing its digits gives back the same number

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	Opt 1: n=4213	Largest possible number = 4321	Largest possible number = 4321	Pass
2	Opt 2: n=4213	Smallest possible number = 1234	Smallest possible number = 1234	Pass
3	Opt 3: n=1234	After swap = 4231	After swap = 4231	Pass
4	Opt 4: n=12321	Output: Palindrome	Output: Palindrome	Pass

Step 3: Algorithm

1. Start the program.
2. Repeat steps 3 to 8 until the user chooses to exit.
3. Display the menu with options: (1) Generate largest number, (2) Generate smallest number, (3) Swap first and last digits, (4) Check palindrome, (5) Exit.
4. Read the user's choice.

5. If choice = 1: call generateLargest(). Inside it, read n, extract its digits into an array, sort the array in descending order, and reassemble the digits into a number by repeatedly multiplying the running result by 10 and adding the next digit; display the result.
6. If choice = 2: call generateSmallest(). Inside it, read n, extract its digits into an array, sort the array in ascending order, and reassemble the digits into a number the same way; display the result.
7. If choice = 3: call swapFirstLast(). Inside it, read n, extract its digits into an array, reverse the array to left-to-right order, exchange the first and last elements, and reassemble the digits into a number; display the result.
8. If choice = 4: call checkPalindrome(). Inside it, read n, build the reversed number by repeatedly extracting the last digit and appending it to a running total, and compare the reversed number with the original; display "Palindrome" if they match, otherwise "Not a Palindrome".
9. If choice = 5: exit the loop and stop the program.
10. If choice is invalid, display an error message and show the menu again.
11. Stop the program.

Evaluation against Step 2 test cases: manually tracing the algorithm above for n = 4213 gives digits {4,2,1,3}; sorted descending (4,3,2,1) reassembles to 4321, and sorted ascending (1,2,3,4) reassembles to 1234; for n = 1234, reversing to left-to-right order (1,2,3,4) and swapping the first and last digits gives (4,2,3,1), reassembling to 4231; for n = 12321, reversing the digits gives 12321 again, so the output is "Palindrome" — identical to the Expected Output(s) column for all 4 test cases, confirming the algorithm is correct before coding.

Step 4: C Program

```
/* Day-17: Menu-driven program using switch-case
Each menu option is implemented as a SEPARATE user-defined function
with void return type, called with a simple function call (no arguments,
no call by value/reference). Each function reads its own input,
processes it, and displays the result.

Option 1: Generate the largest possible number using its digits
Option 2: Generate the smallest possible number using its digits
Option 3: Swap its first and last digits
Option 4: Check whether it is a palindrome number
*/
#include <stdio.h>

// Function Declarations
void generateLargest();
void generateSmallest();
void swapFirstLast();
void checkPalindrome();

int main() {
    int choice;

    do {
        printf("\n----- MENU ----- \n");
        printf("1. Generate the largest possible number using its digits\n");
        printf("2. Generate the smallest possible number using its digits\n");
        printf("3. Swap its first and last digits\n");
        printf("4. Check whether it is a palindrome number\n");
```

```

printf("5. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        generateLargest();
        break;
    case 2:
        generateSmallest();
        break;
    case 3:
        swapFirstLast();
        break;
    case 4:
        checkPalindrome();
        break;
    case 5:
        printf("Exiting program.\n");
        break;
    default:
        printf("Invalid choice. Please try again.\n");
}
} while (choice != 5);

return 0;
}

/* Option 1: Generate largest number using digits */
void generateLargest() {
    int n, temp, digit, count = 0, i, j, t;
    int digits[20];
    long result = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    temp = n;
    while (temp != 0) {
        digit = temp % 10;
        digits[count++] = digit;
        temp /= 10;
    }

    // Sort digits in descending order
    for (i = 0; i < count - 1; i++) {
        for (j = 0; j < count - 1 - i; j++) {
            if (digits[j] < digits[j + 1]) {
                t = digits[j];
                digits[j] = digits[j + 1];
                digits[j + 1] = t;
            }
        }
    }
}

```

```

    for (i = 0; i < count; i++) {
        result = result * 10 + digits[i];
    }

    printf("Largest possible number = %ld\n", result);
}

/* Option 2: Generate smallest number using digits */
void generateSmallest() {
    int n, temp, digit, count = 0, i, j, t;
    int digits[20];
    long result = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    temp = n;
    while (temp != 0) {
        digit = temp % 10;
        digits[count++] = digit;
        temp /= 10;
    }

    // Sort digits in ascending order
    for (i = 0; i < count - 1; i++) {
        for (j = 0; j < count - 1 - i; j++) {
            if (digits[j] > digits[j + 1]) {
                t = digits[j];
                digits[j] = digits[j + 1];
                digits[j + 1] = t;
            }
        }
    }

    // Handle leading zero
    if (digits[0] == 0) {
        for (i = 1; i < count; i++) {
            if (digits[i] != 0) {
                digits[0] = digits[i];
                digits[i] = 0;
                break;
            }
        }
    }

    for (i = 0; i < count; i++) {
        result = result * 10 + digits[i];
    }

    printf("Smallest possible number = %ld\n", result);
}

/* Option 3: Swap first and last digits */
void swapFirstLast() {
    int n, temp, digit, count = 0, i, t;

```

```

int digits[20];
long result = 0;

printf("Enter a positive integer: ");
scanf("%d", &n);

temp = n;
while (temp != 0) {
    digit = temp % 10;
    digits[count++] = digit;
    temp /= 10;
}

// Reverse array to restore left-to-right order
for (i = 0; i < count / 2; i++) {
    t = digits[i];
    digits[i] = digits[count - 1 - i];
    digits[count - 1 - i] = t;
}

// Swap first (index 0) and last (index count - 1) digits
t = digits[0];
digits[0] = digits[count - 1];
digits[count - 1] = t;

for (i = 0; i < count; i++) {
    result = result * 10 + digits[i];
}

printf("After swap = %ld\n", result);
}

/* Option 4: Check whether number is a palindrome */
void checkPalindrome() {
    int n, temp, digit;
    long reversed = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    temp = n;
    while (temp != 0) {
        digit = temp % 10;
        reversed = reversed * 10 + digit;
        temp /= 10;
    }

    if (reversed == n)
        printf("Output: Palindrome\n");
    else
        printf("Output: Not a Palindrome\n");
}

```

Step 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

```
cc hdoc17.c      (compile)
./a.out          (execute)
```

All 4 test cases prepared in Step 2 were executed against the compiled program by selecting the corresponding menu option (each option invoking its own void function with a simple call). The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

```

Session Actions Edit View Help

(kali㉿kali)-[~]
$ nano hdoc17.c

(kali㉿kali)-[~]
$ cc hdoc17.c

(kali㉿kali)-[~]
$ ./a.out

—— MENU ——
1. Generate the largest possible number using its digits
2. Generate the smallest possible number using its digits
3. Swap its first and last digits
4. Check whether it is a palindrome number
5. Exit
Enter your choice: 1
Enter a positive integer: 4213
Largest possible number = 4321

—— MENU ——
1. Generate the largest possible number using its digits
2. Generate the smallest possible number using its digits
3. Swap its first and last digits
4. Check whether it is a palindrome number
5. Exit
Enter your choice: 2
Enter a positive integer: 4213
Smallest possible number = 1234

—— MENU ——
1. Generate the largest possible number using its digits
2. Generate the smallest possible number using its digits
3. Swap its first and last digits
4. Check whether it is a palindrome number
5. Exit
Enter your choice: 3
Enter a positive integer: 1234
After swap = 4231

—— MENU ——
1. Generate the largest possible number using its digits
2. Generate the smallest possible number using its digits
3. Swap its first and last digits
4. Check whether it is a palindrome number
5. Exit
Enter your choice: 4
Enter a positive integer: 12321
Output: Palindrome

—— MENU ——
1. Generate the largest possible number using its digits
2. Generate the smallest possible number using its digits
3. Swap its first and last digits
4. Check whether it is a palindrome number
5. Exit
Enter your choice: 5
Exiting program.
```

○ Conclusion

In this lab session, modular programming with parameterless void functions was utilized to perform positional and structural operations on integer digits:

1. **Option 1 & Option 2 (Digit Sorting & Reconstruction)**: Extracted digits into an array, sorted them via Bubble Sort (descending for maximum, ascending for minimum), and reassembled them back into an integer. Leading zero correction was applied during minimum integer generation to prevent digit truncation.
2. **Option 3 (Digit Positional Swap)**: Reversed the modulo-extracted digit array to restore natural left-to-right digit order, swapped the boundary elements at index 0 and count - 1, and reconstructed the new integer value.
3. **Option 4 (Palindrome Verification)**: Constructed the reverse of the input integer through iterative modulo (% 10) and base-10 positional arithmetic (* 10), directly testing equality against the initial input integer.

All user-defined function calls were managed via a central switch-case construct inside a do-while menu loop. The program compiled cleanly using gcc in Kali Linux, and all actual execution outputs matched the manual test case table predictions.
